Classification of images using various CNN

Gursajal Singh Bagga
The University of Adelaide, Adelaide, Australia
Student ID: a1911873

(**NOTE**: For this study we are using Fashion MNIST Dataset.)

Abstract - In this paper, we explore how different Convolutional Neural Networks (CNNs) perform on the Fashion MNIST dataset. We discuss the importance of image classification in computer vision and explain the methods we used for our experiments. We also compare the results of various CNN models and share what we learned about which ones work best for this dataset. Our findings could help improve image classification tasks and guide the selection of CNN models for similar datasets.

I. INTRODUCTION

A. **Problem Statement**

Image classification is a key challenge in computer vision, with many practical applications in various fields. It involves labeling an image with a category based on its content. Convolutional Neural Networks (CNNs) have become the leading method for solving image classification problems. However, there is still ongoing research into which CNN architectures work best for specific types of datasets.

B**. Data Source**

This study evaluates different CNN architectures using the Fashion MNIST dataset. Fashion MNIST is similar to the popular MNIST dataset but instead of handwritten digits, it contains grayscale images of clothing items. The dataset has 60,000 training images and 10,000 testing images, organized into ten fashion categories. The variety and complexity of Fashion MNIST make it an excellent dataset for testing how well CNNs perform in image classification tasks.

C. **Convolutional Neural Networks (CNN)**

Convolutional Neural Networks (CNNs) are a type of deep neural network designed for processing grid-like data, such as images. They have been highly successful in many computer vision tasks, including image classification. CNNs are made up of multiple layers, each of which extracts and transforms features from the input data. Here's an overview of the key components and functions of CNNs:

1. **Convolution Layer**: The convolutional layer is the most important part of a CNN. It uses learnable filters (also known as kernels) to perform convolution on the input image. Each filter scans the image to detect specific patterns, like edges or textures. The formula is:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot K(i, j)$$

Here, **I** represents the input image, **K** is the kernel (or filter), and **(x, y)** corresponds to a position in the resulting feature map. The indices **(i, j)** iterate over the elements of the kernel as it scans the image.

2. **Activation Function**: After the convolution operation, an activation function is applied to introduce non-linearity into the model. Typically, the ReLU (Rectified Linear Unit) function is used. It is applied element-wise to the output of the convolution layer and is defined as:

$$\mathrm{ReLU}(x) = \max(0, x)$$

3. **Pooling Layer**: The pooling layer reduces the size of the feature maps while retaining important information. It helps to lower computational cost and prevent overfitting. Common types of pooling include max-pooling

and average-pooling. For example, in max-pooling, the maximum value from a local region of the feature map is selected:

MaxPooling(x, y) = max(feature map(x, y))

This operation helps reduce the spatial dimensions of the feature map, making the model more efficient.

4. **Fully Connected Layer:** After passing through several convolution and pooling layers, CNNs typically include one or more fully connected layers. These layers take the flattened output from the previous layer and perform matrix multiplication with a weight matrix. An activation function is then applied to the result. This fully connected layer can be represented as:

$$\text{FC}(x) = \text{Activation}\left(\sum_i \text{Weight}(i) \cdot x(i) + \text{Bias}\right)$$

where **W** is the weight matrix, **x** is the input vector, **b** is the bias, and **activation** refers to the function applied (like ReLU or sigmoid). The fully connected layer helps to classify the extracted features into the final output categories.

5. **Loss Function:** In CNNs, the model is trained using a loss function that measures how far off the predictions are from the true labels. For image classification, the most common loss function is categorical cross-entropy, which calculates the difference between the predicted class probabilities **(p_i)** and the true class labels **(y_i).** The loss function helps the model learn by guiding it to make more accurate predictions.

$$\text{Loss} = -\sum_i y_i \cdot \log(p_i)$$

6. **Backpropagation and Optimization:** CNNs are trained using a process called backpropagation along with optimization methods like stochastic gradient descent (SGD). During training, the model's weights and biases are adjusted to minimize the loss function. This iterative process gradually improves the model's performance. The update rule for the weights can be written as:

Weight(i) = Weight(i) - Learning Rate * Gradient of Loss

The optimization process helps the CNN learn and refine its features over time.

CNNs are especially powerful because they can automatically learn hierarchical features from raw data, which makes them very effective for image classification tasks. The structure of a CNN, including the number of layers, filter sizes, and fully connected layers, can be tailored to suit the specific problem. This flexibility and ability to learn features from the data have made CNNs the model of choice for image classification and similar tasks.

Next, we'll discuss our methodology for evaluating different CNN architectures on the Fashion MNIST dataset.

II. Issues with Convolutional Neural Networks (CNNs)

While CNNs are highly effective, they also come with certain challenges:

1.**Overfitting:** CNNs can sometimes "memorize" the training data rather than generalizing to new, unseen data. This happens when the model is too complex for the available dataset, which leads to poor performance on real-world data.

2. **Training Data Size:** CNNs typically need a large amount of labeled data to train effectively. If the dataset is small or limited, the model may struggle to learn meaningful patterns.

3. **Complexity and Computational Cost:** Deep CNNs with many layers can be computationally intensive, requiring significant resources, such as powerful GPUs or cloud-based computing systems, to train in a reasonable amount of time.

4. **Hyperparameter Tuning**: Finding the right set of hyperparameters (like learning rates, batch sizes, and layer configurations) can be a long and difficult process, often involving trial and error and domain expertise.

5. **Limited Interpretability:** CNNs are often considered "black-box" models, meaning it can be hard to understand how they make decisions or which features they focus on during classification. This lack of transparency can be a barrier in applications where interpretability is important.

6. **Long Training Times:** Training large and deep CNNs can take a long time, especially with large datasets and complex architectures. This can make the training process costly in terms of both time and resources.

7. **Architecture Selection**: Choosing the right architecture for a specific task can be challenging. There are many different types of CNN models, and selecting the best one often requires significant experimentation and expertise in both the dataset and the problem at hand.

III. SOLUTION SUGGESTED

There are several ways of doing the same and few of them are:

A. Batch Normalization (BatchNorm)

Batch Normalization is a technique that normalizes the activations of a neural network layer, typically applied to fully connected and convolutional layers. It has several key benefits, including improved training stability, faster convergence, and regularization.

The Batch Normalization process can be summarized with the following formulas:

For a mini-batch of activations $B = \{x_1, x_2, \ldots, x_m\}$ within a layer:

1. Compute the mean and variance of the mini-batch:

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i,$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2.$$

2. Normalize the activations using the mean and variance:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}},$$

where ϵ is a small constant added for numerical stability.

3. Scale and shift the normalized activations:

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Batch Normalization can be applied at each layer during training, and it can significantly enhance the training process, making it more stable and enabling the use of higher learning rates

B. Dropout (Dropout)

Dropout is a regularization technique used to prevent over- fitting by reducing the reliance on specific neurons during training. It works by randomly setting a fraction of neuron activations to zero during each forward and backward pass [5].

The Dropout process can be described using the following formula:

Given an input value x and a dropout rate p (the probability of setting a value to zero), the dropout operation is defined as:

$$\text{Dropout}(x) = \begin{cases} \frac{x}{1-p}, & \text{with probability } 1-p, \\ 0, & \text{with probability } p. \end{cases}$$

Dropout is applied independently to each neuron's output. During inference (testing or prediction), dropout is usually turned off, and the model scales the neuron activations by a factor of $1 - p$ to ensure the expected output remains the same.

By introducing noise and reducing the interdependencies between neurons, Dropout helps prevent overfitting and im- proves the generalization of neural networks.

There are many benefits of Dropout. Some of them are:

1.**Overfitting Prevention:** Dropout helps prevent overfitting by adding randomness during training. It forces the model to rely less on specific neurons, preventing them

from "co-adapting" to the training data and encouraging more robust learning.

2.**Enhanced Generalization**: By randomly deactivating neurons, Dropout makes the model less reliant on any one feature or pattern. This encourages the network to learn more general, meaningful features, which improves its ability to perform well on unseen data.

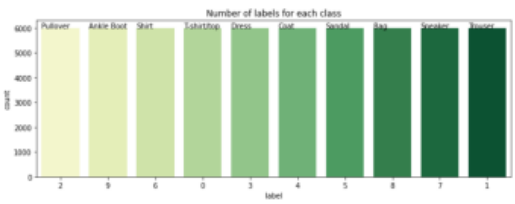
3.**Improved Model Robustness:** Models trained with Dropout are typically more resilient to noisy or imperfect input data. Since the network is less likely to focus on specific patterns, it's better at handling minor variations in the data without making overly confident or incorrect predictions.

IV. DATA ANALYSIS

In this section, we present an analysis of the dataset to gain insights into the relationships between various parameters. We employ a correlation matrix to assess the associations between features and utilize box plots to visualize data distributions.

A. **Data Exploration**

Analysing the class distribution is essential to gain insights into the balance of the Fashion MNIST dataset. This analysis helps determine if there are enough examples for each clothing category and if the dataset is suitable for training and evaluation.



Class Distribution

The above figure shows the class distribution in the Fashion MNIST dataset. Each class corresponds to a different clothing category, such as T-shirts, dresses, or shoes. The y-axis represents the number of examples in each class, while the x-axis shows the class labels.

Key Observations from the Class Distribution:

1. The dataset appears to be well-balanced, with roughly an equal number of examples for each clothing category.
2. In situations where data imbalance is an issue, techniques like oversampling, under sampling, or class-weighted loss functions can be used to address the problem and ensure fair model training.

V. DATA PREPROCESSING

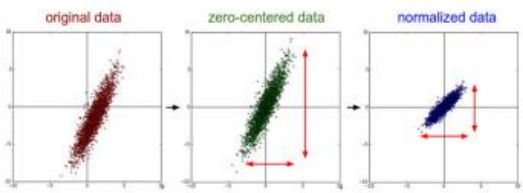
Data preprocessing is an essential step for preparing the Fashion MNIST dataset for machine learning tasks. In this section, we describe the specific preprocessing steps applied to the raw data to make it ready for training and evaluation.

A. **Normalization**

The pixel values in the Fashion MNIST dataset range from 0 to 255. To help the machine learning models converge more effectively and improve their performance, we applied normalization to rescale these values to a consistent range, typically between 0 and 1. This step ensures that the values are on a comparable scale, making it easier for the model to learn.

The normalization process was carried out using the following formula for each pixel value x:

$$x_{\text{normalized}} = \frac{x}{255}$$



Class Distribution Normalized

This formula scales each pixel value x (which originally ranges from 0 to 255) to a new value between 0 and 1.

B. **Data Splitting: Training and Validation Sets**

The original Fashion MNIST dataset contains 60,000 training images and 10,000 testing images. To properly evaluate the model's performance and avoid issues like data leakage, we split the dataset into a training set, a validation set, and the testing set. The typical split used is as follows:

1. Training Set: 80% of the data
2. Validation Set: 20% of the data

This splitting approach ensures that the model is trained on one portion of the data, evaluated on unseen data during training (the validation set), and finally tested on a separate testing set to measure its generalization ability.

C. **Label Encoding**

The labels in the Fashion MNIST dataset are integers ranging from 0 to 9, each corresponding to a specific clothing category. For machine learning classification tasks, we need to convert these integer labels into a format suitable for multi-class classification. To do this, we applied one-hot encoding.

One-hot encoding transforms each label into a binary vector, where only one element is "1" (indicating the correct class), and all other elements are "0". This format allows the model to handle multiple classes and is commonly used in classification tasks.

The one-hot encoding process for each label y (with K possible categories) can be represented as:

$$\mathbf{y_{one\text{-}hot} = [0, 0, ..., 1, ..., 0]}$$

Here, $y_{one\text{-}hot}$ is the one-hot encoded label, which is a binary vector of length K, where all values are 0 except for the one corresponding to the correct category. A "1" is placed at the index corresponding to the category, and all other positions are filled with 0s.

This encoding allows the model to treat each class independently and is ideal for multi-class classification problems like the one in Fashion MNIST.

VI. EXPERIMENTS AND RESULTS

In this section, we present the experiments we conducted to evaluate our proposed method. The primary goal is to demonstrate how effective our approach is and to analyze how different factors influence its performance. To achieve this, we carried out a series of experiments using three different types of Convolutional Neural Network (CNN) architectures. The aim was to identify which architecture performs the best on the Fashion MNIST dataset.

These experiments help us better understand the impact of various design choices, such as network depth, filter sizes, and other hyperparameters, on the model's ability to classify fashion items accurately. The results from these experiments will guide us in selecting the optimal CNN architecture for the task at hand.

The 4 CNN architecture we are going to evaluate are:

A. Different CNNs

Fig. 3: Model 1

Fig. 4: Model 2

Fig. 5: Model 3

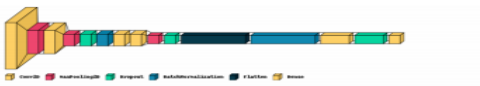


Fig. 6: Model 4

The results of our experiments clearly highlight the positive impact of Dropout and Batch Normalization on model performance. Models that incorporated both Dropout and Batch Normalization consistently outperformed models where either technique was omitted, showing higher test accuracy and lower test loss.

1. Batch Normalization alone helped improve the model's performance, but the benefits were more pronounced when combined with Dropout.
2. Dropout, by introducing randomness into the training process, proved effective in preventing overfitting and improving the model's ability to generalize to unseen data.

However, it is important to note the trade-off between model complexity and performance. For instance, using a very high Dropout rate can sometimes reduce accuracy due to excessive regularization. This demonstrates the need for careful tuning of hyperparameters to strike the right balance.

The specific impact of Dropout and Batch Normalization may vary depending on the architecture, dataset, and problem domain, making experimentation crucial to finding the optimal configuration for a given task.

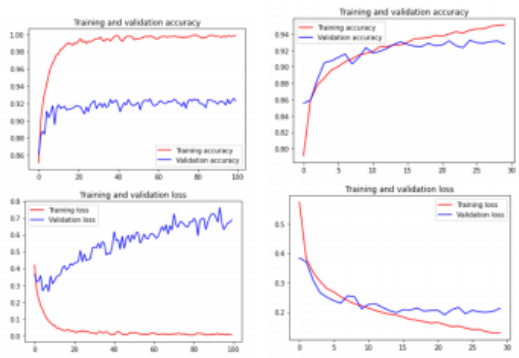
Ultimately, Model 4 demonstrated the best overall performance, so we plan to conduct further experiments with this model to optimize its efficiency and explore ways to enhance its performance even further.

Comparison Table:

TABLE I: Comparison of Model Performances

Model	Configuration	Test Loss	Test Accuracy
Model 1	With Dropout and Batch Normalization	0.1848	0.9370
	Without Batch Normalization	0.2472	0.9081
	Without Dropout	0.4896	0.9199
	Without Batch Normalization and Dropout	0.4435	0.9015
Model 2	With Dropout and Batch Normalization	0.3076	0.9312
	Without Batch Normalization	0.2673	0.9210
	Without Batch Normalization and Dropout	0.8375	0.8877
	Without Dropout	0.4092	0.9252
Model 3	With Batch Normalization and Dropout	0.2085	0.9287
	Without Batch Normalization	0.2057	0.9264
	Without Batch Normalization and Dropout	0.4978	0.9127
	Without Dropout	0.4484	0.9198
Model 4	With Dropout and Batch Normalization	0.2306	0.9423
	Without Batch Normalization	0.1867	0.9323
	Without Batch Normalization and Dropout	0.5750	0.9195
	Without Dropout	0.3442	0.9383

B. **Loss and Accuracy**



(a) Accuracy and Loss, Without Dropout (b) Accuracy and Loss, With Dropout

Comparison of Models with and without Dropout

Conclusions: Increasing the number of training epochs improves accuracy and reduces loss for validation and training data set upto a certain epoch, but than it starts to increase the validation loss. It can be noted that the validation accuracy and loss is greatly improved when using dropout to optimize the CNN.

C. Using Different Optimizers

Motivation: The motivation is to discover the optimizer that maximizes the performance of deep learning models on image classification. Optimizers influence convergence and accuracy. By exploring options like Adam, Adagrad, Nadam, RMSprop, SGD, and Ftrl, we aim to optimize neural networks for real- world applications.

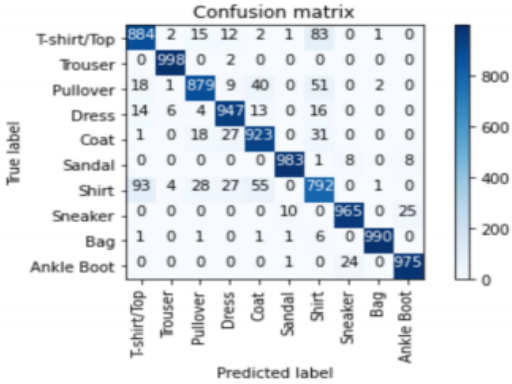
TABLE II: Comparison of Different Optimizers

Optimizer	Test Loss	Test Accuracy
Adam	0.2306	0.9423
Adagrad	0.3326	0.8767
Nadam	0.1948	0.9351
RMSprop	0.2559	0.9217
SGD	0.2677	0.9014
Ftrl	2.3026	0.1000

VII. RESULT ANALYSIS

TABLE III: Classification Report

Class	Precision	Recall	F1-Score
Class 0	0.88	0.87	0.88
Class 1	0.99	1.00	0.99
Class 2	0.94	0.87	0.90
Class 3	0.94	0.94	0.94
Class 4	0.90	0.92	0.91
Class 5	0.99	0.98	0.99
Class 6	0.82	0.78	0.80
Class 7	0.97	0.96	0.97
Class 8	1.00	0.99	0.99
Class 9	0.97	0.97	0.97
Micro Avg	0.94	0.93	0.93
Macro Avg	0.94	0.93	0.93
Weighted Avg	0.94	0.93	0.93
Samples Avg	0.93	0.93	0.93



Conclusions: Among the tested optimizers, Adam achieved the highest test accuracy of 0.9423, indicating effective con- vergence and model optimization.

• The model exhibits strong classification capabilities across multiple classes, with Class 1 achieving near- perfect precision, recall, and F1-score, reflecting accurate predictions.

• Class 2 displays a good level of precision and F1-score but has a slightly lower recall, indicating potential for improvement in identifying true positives.

• The macro and weighted average F1-scores are consis- tent at 0.93, indicating the model's generalization and balanced performance across various classes.

• The samples average F1-score of 0.93 underscores the model's ability to achieve reliable classification results for individual instances within the dataset.

From Confusion Matrix it can be concluded that a lot of Shirts are being identified as T-Shirt.

VIII. CODE

We provide the Python code for our implementation and Experiments on CNN on GitHub:https://github.com/gursajalsingh/CNN_image_classification

IX. CONCLUSION

In this study, we explored a variety of Convolutional Neural Network (CNN) architectures for image classification on the Fashion MNIST dataset. Our findings revealed that different CNN architectures yield varying levels of accuracy and efficiency. This research contributes to a deeper understanding of the impact of architecture choices on model performance, serving as a valuable resource for practitioners seeking to optimize their image classification solutions.

• Learning Rate Decay: Using Learning rate decay so that every epoch the learning rate to decrease overfitting the model to test data.

• Regularization Techniques: Further research into advanced regularization methods, such as capsule networks or attention mechanisms, may help enhance model robustness.

• Transfer Learning: Exploring the potential of transfer learning by fine-tuning pre-trained models on the Fashion MNIST dataset can offer insights into leveraging external knowledge.

• Data Augmentation: Enhancing data augmentation strategies can aid in improving model generalization and performance.

• Different Loss function: Experimentations can be done by trying different loss function

X. REFERENCES

[1] Dozat, T. (2016). Incorporating nesterov momentum into adam. In International Conference on Learning Represen- tations Workshops.

[2] Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. arXiv preprint arXiv:1502.03167.

[3] Karpathy, A. (2014). Cs231n convolutional neural net- works for visual recognition. Accessed on: Date you accessed the course.

[4] Kingma, D. P. and Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.

[5] Xiao, H., Rasul, K., and Vollgraf, R. (2017). Fashion- mnist: a novel image dataset for benchmarking machine learning algorithms. Accessed on: Date you accessed the dataset.